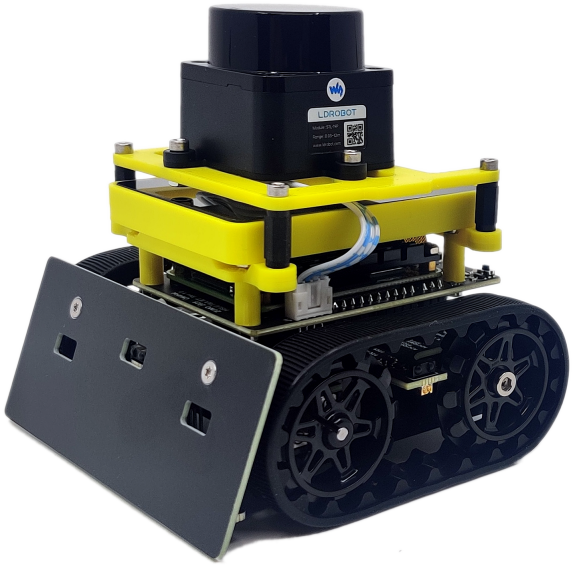
HSLU Hochschule Luzern

CSA
C# in Action

.NET auf Raspberry Pi / Zumo



1. März 2026

Christian Jost

Inhaltsverzeichnis

1 Zumo (Raspberry Pi Compute Module 4 aufsetzen) 2
1.1 Vorbereitungen . . . 2
1.1.1 Raspberry Pi Bootloader . . . 2
1.1.2 Treiber für USB-to-Serial Wandler . . . 2
1.2 Flash-Speicher als Laufwerk einbinden . . . 3
1.3 Linux Image auf die SD-Karte kopieren . . . 4
1.4 Konfigurationseinstellungen auf der SD-Karte . . . 7
1.5 Terminal via UART (USB-to-serial) Verbindung . . . 8
1.6 WLAN Verbindung . . . 10
1.7 Host Namen . . . 15
1.8 SSH Verbindung . . . 16
1.9 Passwort . . . 17
1.10 WinSCP . . . 17
1.11 Updates . . . 18

2 Linux Adminstration 19
2.1 Nützliche Linux Kommandos . . . 19
2.2 Backup der SD-Karte (nicht für den Zumo) . . . 20
2.3 History löschen . . . 22

3 SSH Verbindung ohne Passworteingabe 23
3.1 Schlüssel auf Linux Seite generieren . . . 23
3.2 Windows Host Konfiguration . . . 24
3.3 Testen der Verbindung . . . 25
3.4 SSH ohne Passwort mit PuTTY . . . 26

4 .NET Cross Development Installation auf Linux 29
4.1 Installation von .NET auf Linux . . . 29

1 Zumo (Raspberry Pi Compute Module 4 aufsetzen)

Im Zumo Roboter ist ein Raspberry Pi Compute Module 4 verbaut. Hierbei handelt es sich um ein Modul, auf dem ein Raspberry Pi 4 Mikrocontroller mit Flash, RAM und WLAN aufgelötet sind.

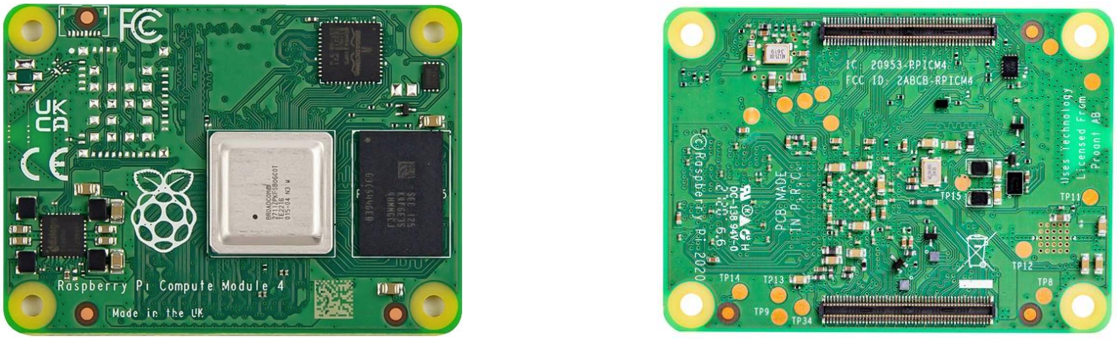


Abbildung 1: Raspberry Pi Compute Module Vor-/Rückseite

Mit Hilfe dieser Anleitung können Sie das Raspberry Pi aufsetzen und per WLAN ins Hochschulnetz und ins private WLAN einbinden.

1.1 Vorbereitungen

1.1.1 Raspberry Pi Bootloader

Beim Compute Module ist der Flash Speicher aufgelötet und kann nicht wie bei einem Raspberry Pi entfernt werden. Deshalb wird das Linux-Image via USB auf das Raspberry Pi geladen. Dazu wird folgendes USB-Verbindungstool verwendet:

Für Windows wird die Datei rpiboot_setup.exe installiert:
https://github.com/raspberrypi/usbboot/tree/master/win32

Für Linux/macOS gibt es folgende Anleitung:
https://github.com/raspberrypi/usbboot/tree/master

1.1.2 Treiber für USB-to-Serial Wandler

Um das WLAN einzurichten, wird ein Zugriff auf die Linux Konsole benötigt. Dies kann mittels serielle Schnittstelle erfolgen. Den dazu benötigten Treiber (JLink Software and Documentation pack) findet man auf folgender Seite: https://www.segger.com/downloads/jlink/

1.2 Flash-Speicher als Laufwerk einbinden

Damit der Speicher des Raspberry Pi beschrieben werden kann, wird dieser als Laufwerk ins Betriebssystem des Notebook eingebunden.

1. Verbinden Sie das Ladegerät des Zumo Roboters mit der unteren, mittleren USB-C Buchse.
2. Den Zumo Roboter einschalten, indem der Power-Taster kurz nach rechts gedrückt wird. Ihn anschliessend ganz nach links schieben (Console On).
3. Mit einem USB-C Kabel das CM4-Board mit dem Notebook verbinden (Anschluss CM4 USB-C).
4. Das CM4-Board einschalten, indem der Taster oben rechts (CM4 Power on/off) etwa 6 Sekunden lang gedrückt wird. Nach ca. 2 Sekunden geht eine gelbe LED neben dem Taster an. Nach ein paar Sekunden erlischt sie wieder. Dies ist das Zeichen, dass man nun den Taster loslassen kann.

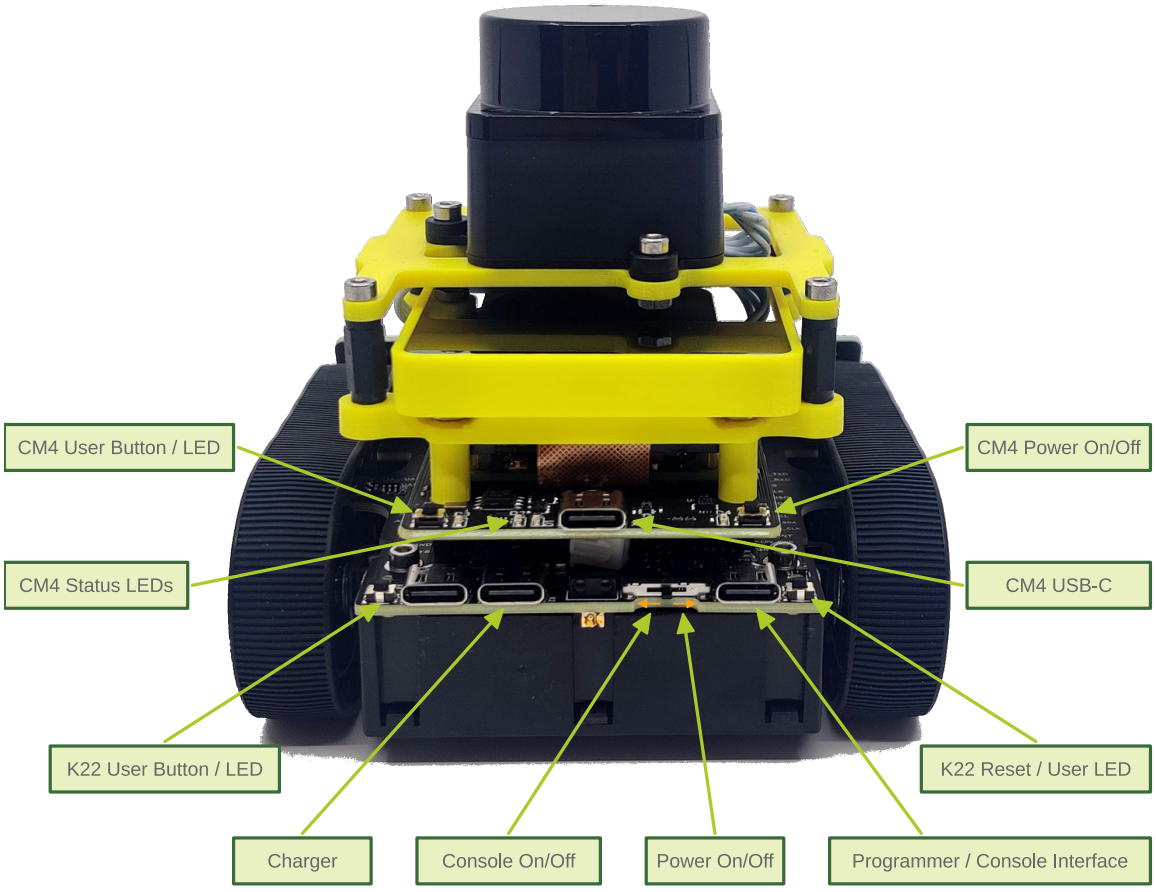


Abbildung 2: Zumo Robot

5. Auf dem CM4 Board sollte nun eine rote Led neben der USB-C Buchse leuchten. Nun kann das Tool ”Raspberry Pi - Mass Storage Gadget - 64-bit” gestartet werden und etwa 30 Sekunden später sollte ein neues Laufwerk (Mass Storage Device) im Betriebssystem sichtbar werden. Falls Windows den Speicher formatieren möchte, können sie dies ablehnen (Cancel/Abbrechen).

1.3 Linux Image auf die SD-Karte kopieren

Im nächsten schritt wird das Betriebssystem (Raspberry Pi OS) auf den Flash Speicher kopiert. Am einfachsten geht dies mit dem Raspberry Pi Imager.

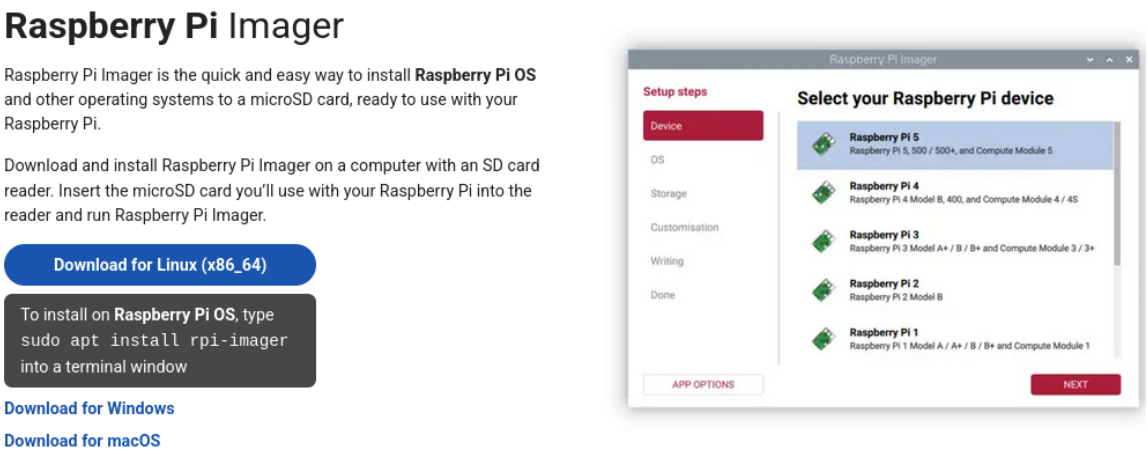


Abbildung 3: Raspberry Pi OS Imager Download

Info: Die nachfolgenden Anleitungen und Codebeispiele sind getestet und verifiziert mit der Version: '2025-12-04 Raspberry Pi OS Lite (64-bit)'.

1. Laden Sie den Raspberry Pi OS Imager von der offiziellen Webseite herunter: Download Raspberry PI OS. Installieren und starten sie das Programm. Unter Linux als Administrator.
2. Wählen Sie bei 'Device' das Raspberry Pi 4 aus.

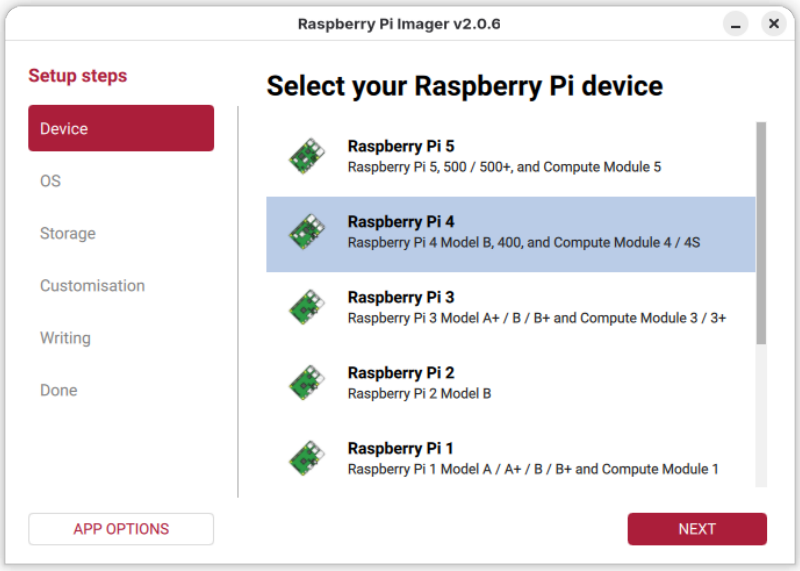


Abbildung 4: Raspberry Pi 4

3. Wählen Sie beim Imager Programm das Raspberry Pi OS Lite (64-bit). Dazu nach unten scrollen und den Menüpunkt 'Raspberry Pi OS (other)' auswählen.

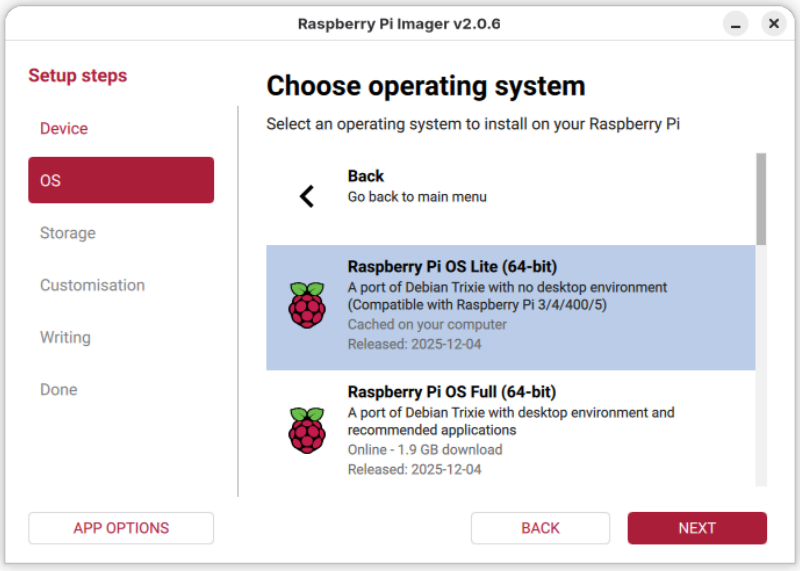


Abbildung 5: Raspberry Pi OS Image

4. Wählen Sie nun die virtuelle Speicherkarte vom Zumo Roboter aus (RPi-MSD-0001). Dieser hat eine Grösse von 29.1 GB

5. Im nächsten Schritt kann das Image individuell angepasst werden. Als erstes wird der Hostname gesetzt. Hier den Namen ’raspberrypi’ verwenden:

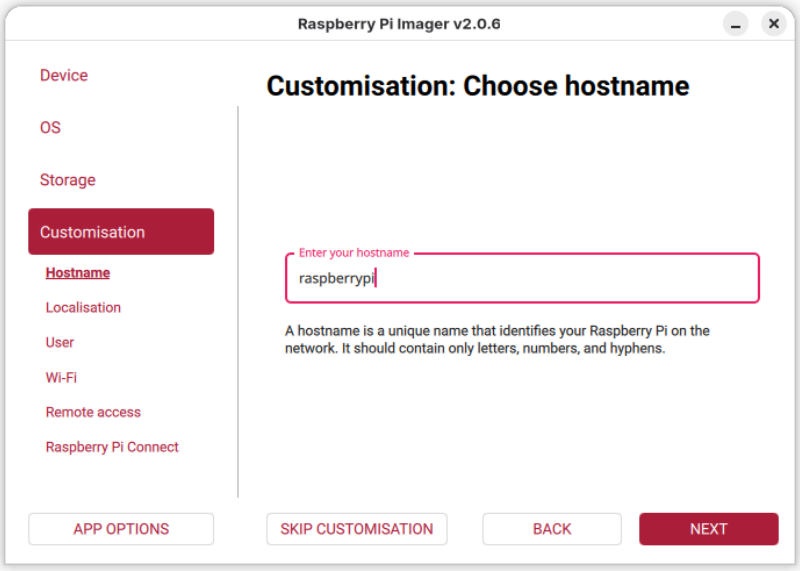


Abbildung 6: Imager Settings

Die weiteren Einstellungen wie folgt wählen:

- ’Localisation’:
  - Capital city: Bern (Switzerland)
  - Time zone: Europe/Zurich

  - Keyboard layout: ch
- ’User’: Wählen Sie als Benutzer ’pi’ sowie ein Passwort. Verwenden Sie nur Zeichen aus den ersten 127 Zeichen der ASCII Tabelle. Sie können das Passwort später noch ändern.
- ’Wi-Fi’: Hier keine Einstellungen vornehmen.
- ’Remote access’: Aktivieren Sie ’Enable SSH’ und wählen Sie ’Use password authentication’
- ’Raspbery Pi Connect’: deaktivieren

6. Bei den ’APP OPTIONEN’ alles deaktivieren.
7. Starten Sie nun das Beschreiben des Flash Speichers mit ’WRITE’. Dieser Vorgang dauert ein paar Minuten.

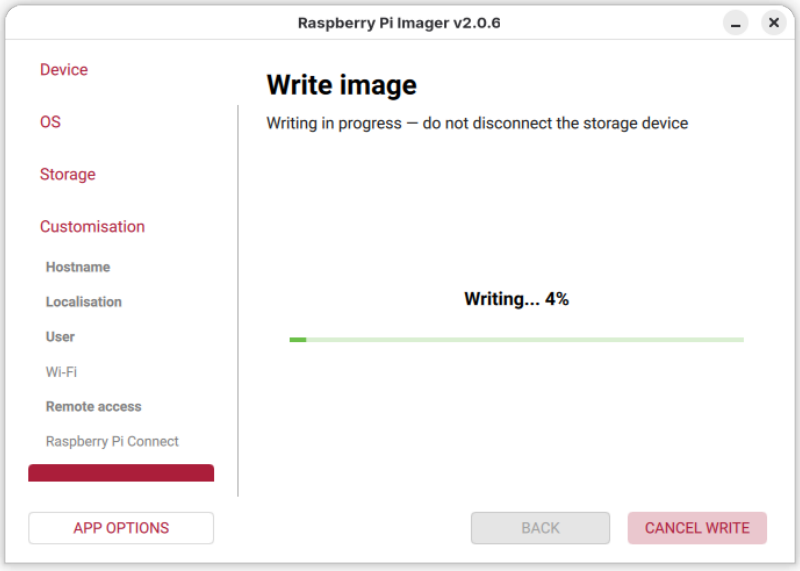


Abbildung 7: Imager Write

8. Nach dem Beenden des Schreibvorgangs kann das Tool beendet werden. Das USB Kabel nicht entfernen und das neue Laufwerk nicht auswerfen!

1.4 Konfigurationseinstellungen auf der SD-Karte

Vor dem ersten Start muss die UART-Schnittstelle konfiguriert werden, damit nach dem Bootvorgang mittels serieller Schnittstelle auf die Konsole zugegriffen werden kann.

1. Nach dem Schreibvorgang sieht man im Windows Datei Explorer zwei neue Laufwerke. Nur das Laufwerk mit der Bezeichnung 'bootfs' ist lesbar. Hier werden wir nun die Konfiguration anpassen. Auf keinen Fall das andere Laufwerk vom Betriebssystem formatieren lassen! Unter Linux sollte ebenfalls ein Laufwerk 'bootfs' sichtbar sein.

2. Ersetzen Sie die Dateien config.txt, network-config sowie user-data mit den abgegebenen Dateien.

3. Werfen Sie die SD Karte 'kontrolliert' aus dem PC aus ('Eject generic SD card' oder ähnlich).

4. Das CM4 Board kann nun ausgeschaltet werden, indem etwa 6 Sekunden auf den CM4 Power on/off Knopf gedrückt wird. Sobald die rote Led nicht mehr leuchtet, ist es ausgeschaltet.

5. CM4 Board wieder einschalten. Diesmal den Taster nur solange drücken, bis die gelbe Led anfängt zu leuchten, da es nicht wieder in den Bootloader Modus versetzt werden soll.

6. Das CM4 Board startet nun das Linux Betriebssystem. Die Zugriffe auf den Flash-Speicher sieht man durch das leuchten/flackern der grünen Led. Das CM4-Board startet etwa 2x neu bis man sich nach ca. 1-2 Minuten einloggen kann.

1.5 Terminal via UART (USB-to-serial) Verbindung

Als erstes verbinden wir uns über die vorgängig konfigurierte UART Verbindung, um das Netzwerk zu konfigurieren und die MAC Adresse herauszufinden.

> **Hinweis:** Es wird das Tool "PuTTY" empfohlen. Andere Terminal Programme funktionieren unter Umständen nicht korrekt (Line Endings, Echo, Sonderzeichen).

1. Verbinden Sie das USB Kabel mit der Konsolen USB-C Buchse (unten rechts auf dem Zumo Roboter - siehe Abbildung 8).

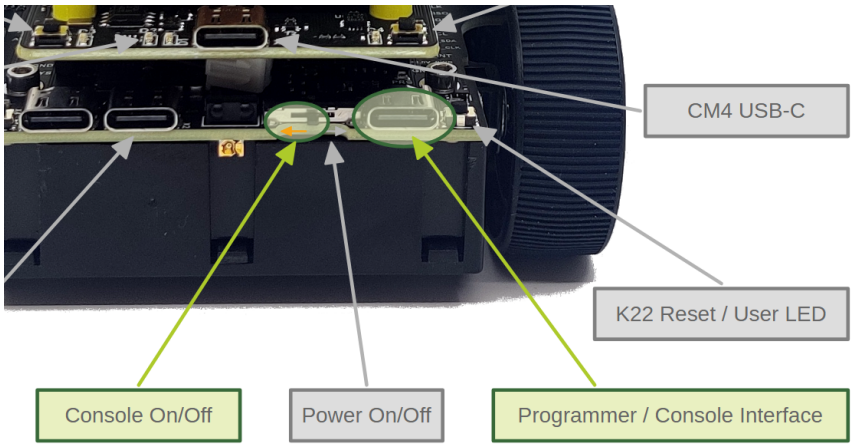


Abbildung 8: Zumo Konsolenanschluss

2. Im Geräte-Manager von Windows sollte nun ein neuer COM Port ersichtlich sein. Es sollte sich um einen JLink Port handeln. Merken Sie sich den COM Port (Nummer). In der Abbildung 9 handelt es sich im Beispiel um den COM Port 13 (COM13). Unter Linux kann mit 'sudo dmesg' geprüft werden, wie die Schnittstelle heisst. Evtl. '/dev/ttyACM0'.

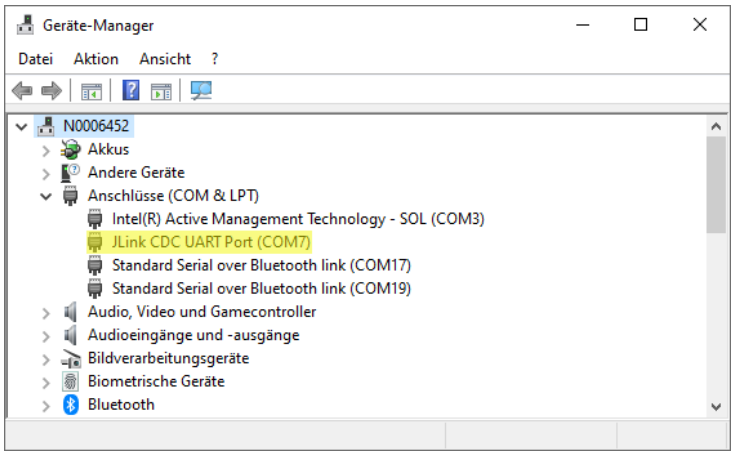


Abbildung 9: JLink CDC UART Port in Device Manager

3. Mit einem Terminal Programm (z.B. 'Putty', Abbildung 10) kann man sich nun auf das Raspberry Pi verbinden. Hier müssen Sie u.a. den korrekten COM Port verwenden! Die Einstellung kann mit Save unter einem Namen gespeichert werden. Achtung, Es kann sein, dass wenn Sie das USB Kabel beim nächsten Mal in einen anderen USB Port an Ihrem Notebook einstecken, Ihnen eine andere COM Port Nummer zugewiesen wird.

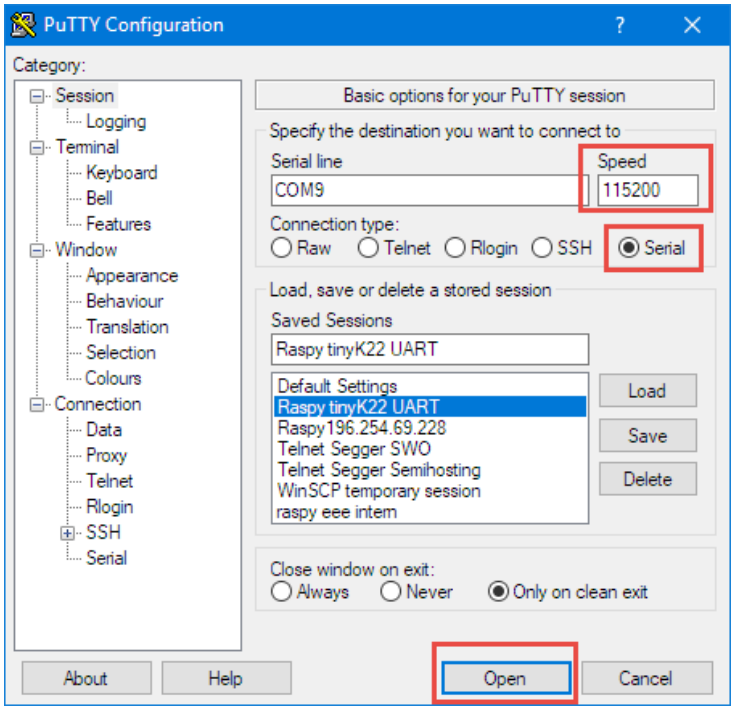


Abbildung 10: Serielle Verbindung mit PuTTY

> **Info:** Im PuTTY die `ENTER` Taste drücken, um den Login Prompt zu bekommen.

4. Nach dem Betätigen der Enter-Taste sollten Sie den Login Prompt sehen und sich mit den Login-Daten einloggen, die sie bei der Erstellung des Images angegeben haben.

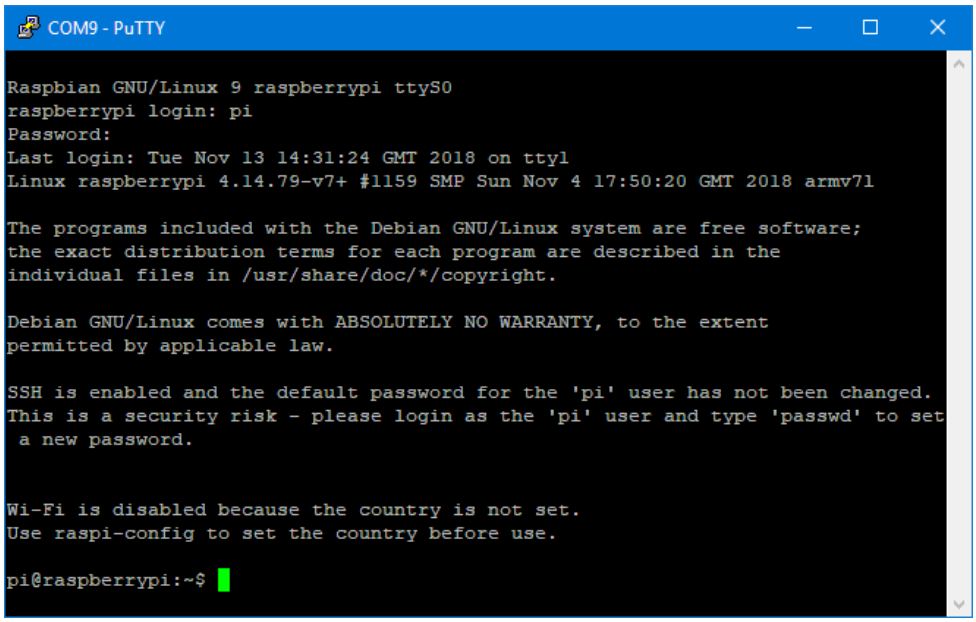


Abbildung 11: Logging mit PuTTY über UART

1.6 WLAN Verbindung

Im nächsten Schritt soll die WLAN Verbindung hergestellt werden. Die serielle Verbindung via USB Kabel wird anschliessend nur noch in Ausnahmefällen benötigt, um Verbindungsprobleme zu lösen.

1. Standardmässig ist das WLAN deaktiviert bis der Standort gesetzt wurde. Starten Sie dazu raspi-config:

```
pi@raspy:$ sudo raspi-config
```

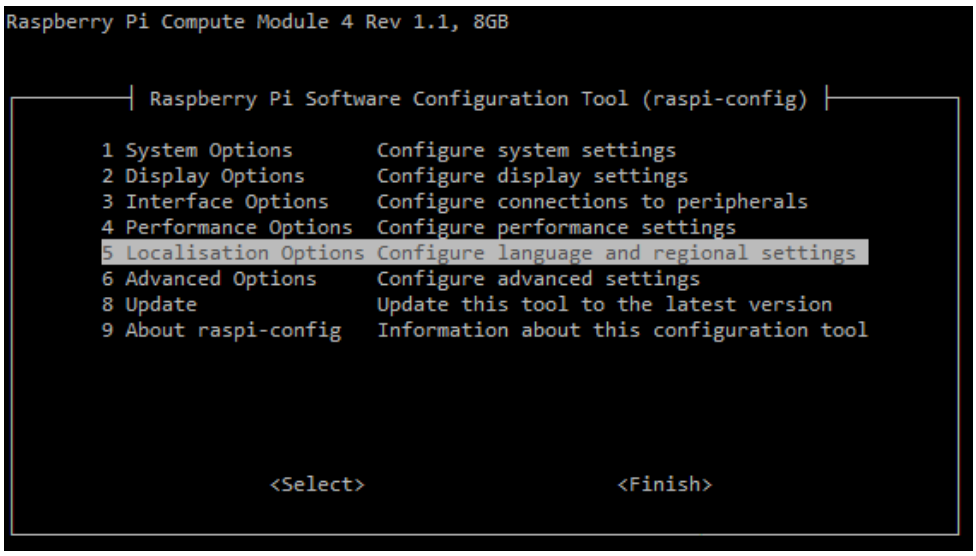


Abbildung 12: WLAN Konfiguration I

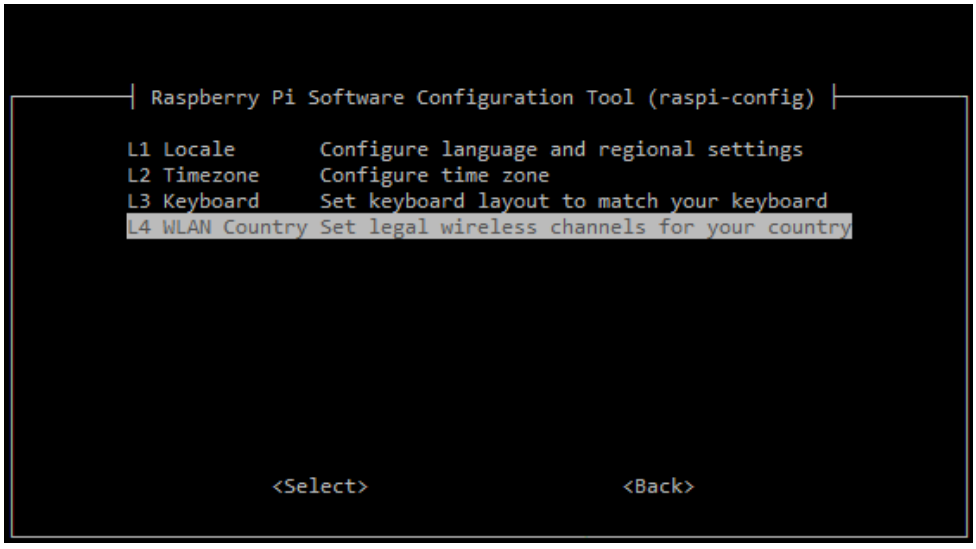


Abbildung 13: WLAN Konfiguration II

Anschliessend das Raspberry Pi neu starten:

```
pi@raspy:$ sudo reboot
```

2. Nun können Sie die MAC-Adresse des WLAN Transceivers (***wlan0***) wie folgt bestimmen

```
pi@raspy:$ ifconfig
```

```
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.180.254.53  netmask 255.255.254.0  broadcast 10.180.255.255
        inet6 fe80::58dd:209f:349b:5621  prefixlen 64  scopeid 0x20<link>
        ether b8:27:eb:[redacted]  txqueuelen 1000  (Ethernet)
        RX packets 71  bytes 9475 (9.2 KiB)
```

Abbildung 14: MAC Adresse wlan0

3. Loggen Sie sich im nächsten Schritt auf https://eeeportal.hslu.ch/ ein und erfassen Sie ein neues Gerät mit der vorher bestimmten MAC Adresse (Abbildung 15).

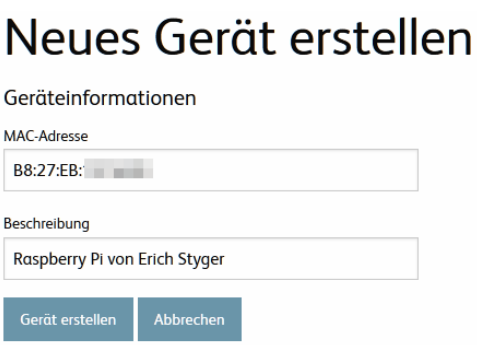


Abbildung 15: Neues Gerät Erstellen

4. Dem neuen Gerät wird ein Benutzername und ein Passwort zugwiesen, die sie in der Geräteliste einsehen können. Mit 'Lease prüfen' kann man später bei erfolgter Anmeldung die IP Adresse abfragen.

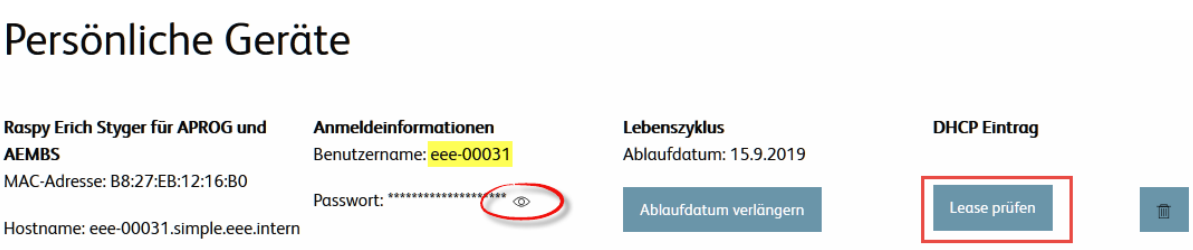


Abbildung 16: Persönliches Gerät

5. Starten Sie den Netzwerk Manager mit dem folgenden Befehl:

```
pi@raspy:$ sudo nmtui
```

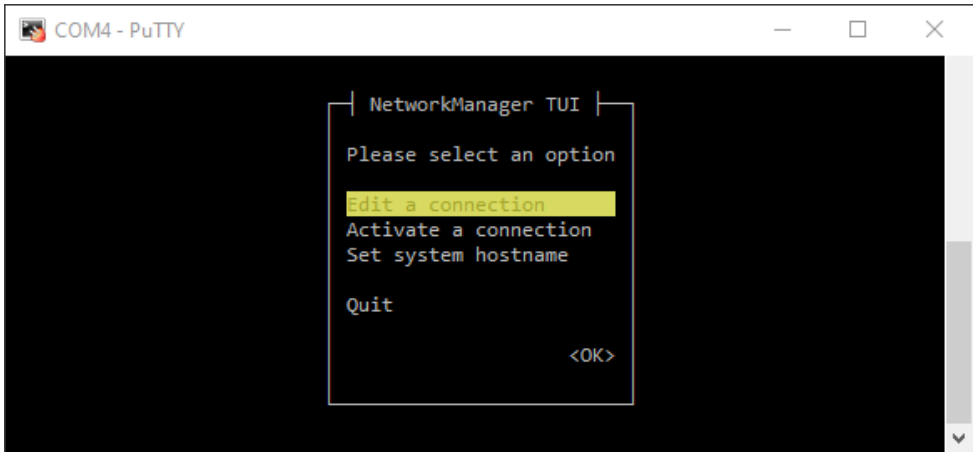


Abbildung 17: Network Manager

6. Wählen Sie ’Edit a connection’ und fügen Sie mit ’Add’ eine neue WiFi Verbindung hinzu:

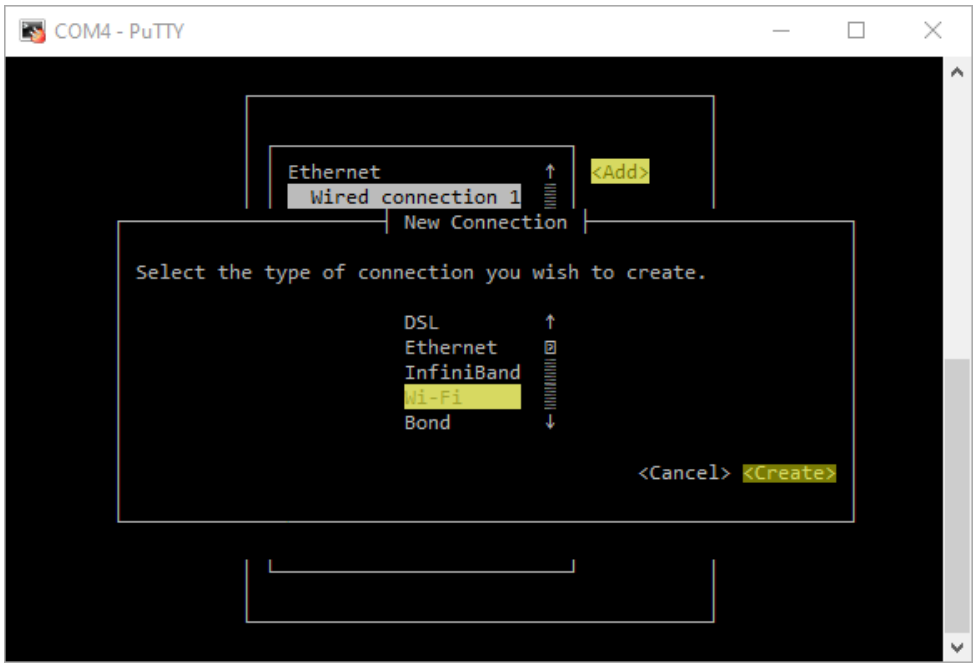


Abbildung 18: WiFi Konfiguration erstellen

7. Im ’Edit Connection’ Dialog nehmen Sie folgende Einstellungen vor:

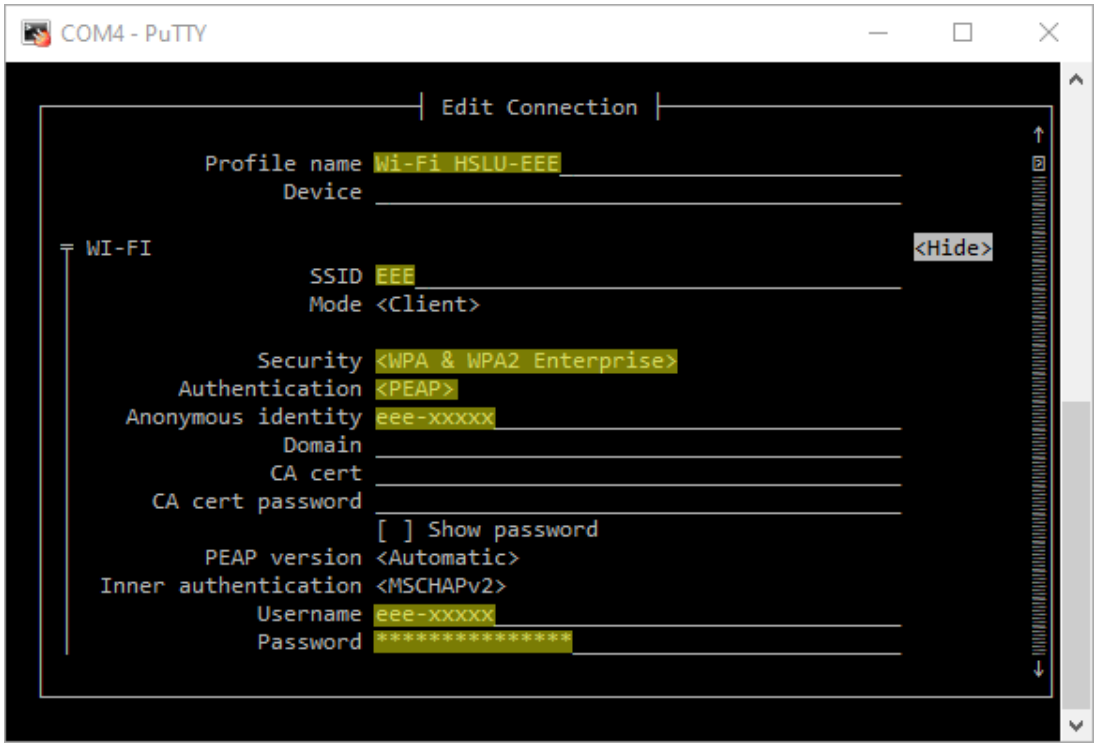


Abbildung 19: Edit Connection

- Als Profilname geben Sie 'Wi-Fi HSLU-EEE' ein
- Als SSID geben Sie 'EEE' ein
- Bei Security wählen Sie <WPA & WPA2 Enterprise>
- Bei Authentication wählen Sie <PEAP>
- Bei 'Username' und 'Password' geben Sie die Ihnen im EEE-Portal zugewiesenen Anmeldeinformationen ein
- Alle anderen Felder frei bzw. auf den Standardeinstellungen belassen
- Scrollen Sie mit den Cursor-Tasten nach unten auf 'OK' und bestätigen Sie mit Enter.

8. Danach gehen Sie mit <Back> ins Hauptmenü zurück und wählen <Activate a connection>
9. Wählen Sie 'Wi-Fi HSLU-EEE' und aktivieren Sie die Verbindung mit Enter
10. Verlassen Sie den Dialog mit <Back> und anschliessend auch das Hauptmenü mit <Quit>
11. Überprüfen Sie, dass eine IP Adresse vergeben wurde (Abbildung 20). Es kann unter Umständen eine Minute dauern, bis man eine IP das erste Mal erhält.



Abbildung 20: DHCP Eintrag

12. Damit sollten Sie WLAN Zugriff zum Internet haben. Mit dem Befehl `ping` kann überprüft werden ob man eine Verbindung zur entsprechenden Webseite aufbauen kann. (`Ctrl`+`C` um ping zu beenden).

```
pi@raspy:$ ping www.google.com
```

13. Mit dem Befehl

```
pi@raspy:$ ifconfig
```

(**inter**face **config**uration) kann man die eigene WLAN IP-Adresse herausfinden. Alternativ kommt man auch via EEE-Portal an die Information (Abbildung 21).

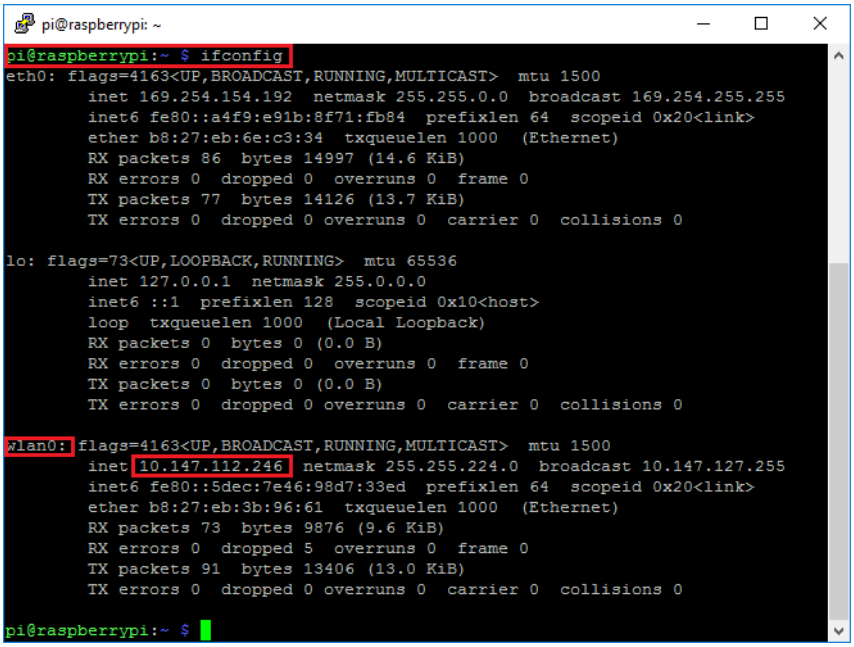


Abbildung 21: Befehl ifconfig

14. Falls Sie ein LAN Kabel verwenden, können Sie ebenfalls mit dem Befehl `ifconfig` die IP Adresse des Raspberry Pi herausfinden. Die Abbildung 22 zeigt die IP Adresse bei Verwendung des Netzwerk-Kabels (**eth0**).

```
pi@raspy:$ ifconfig
```

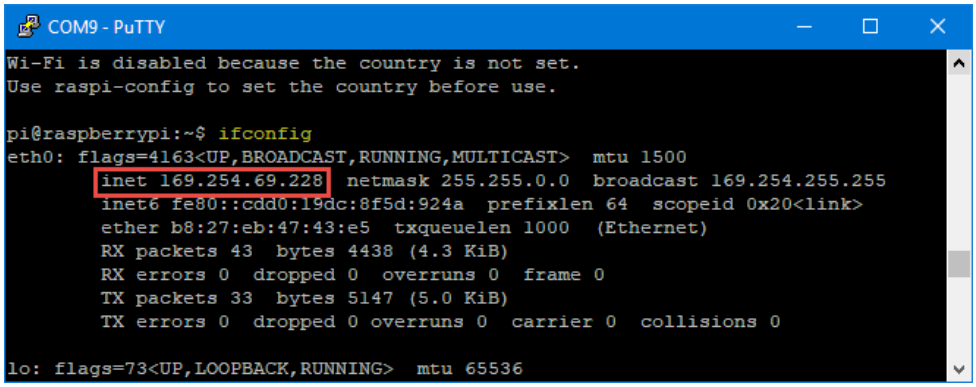


Abbildung 22: IP Adresse

15. Nun können Sie mit Putty eine SSH Verbindung zum Raspberry Pi aufbauen, indem Sie die IP Adresse von Ihrem Raspberry Pi als Hostnamen verwenden.

16. Zur Überprüfung der Netzwerkverbindung können Sie das Raspberry Pi vom PC aus 'anpingen' (Abbildung 23):

```
PC> ping <ip>
```

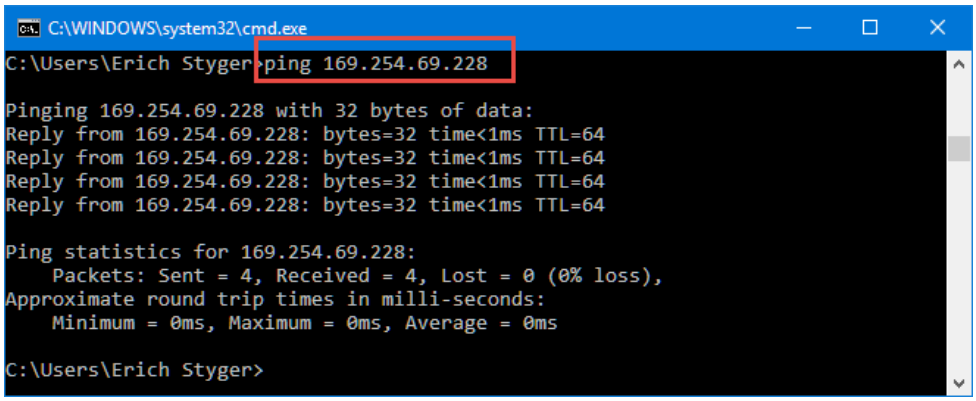


Abbildung 23: Ping vom Host

1.7 Host Namen

1. Standardmässig wird der Hostname 'raspberrypi' verwendet. Den verwendeten Namen können Sie mit

```
pi@raspy:$ hostname
```

überprüfen. Dieser muss im nächsten Schritt auf den Namen geändert werden, der Ihnen im EEE-Portal zugewiesen wurde (Abbildung 24): **ohne** simple.eee.intern.

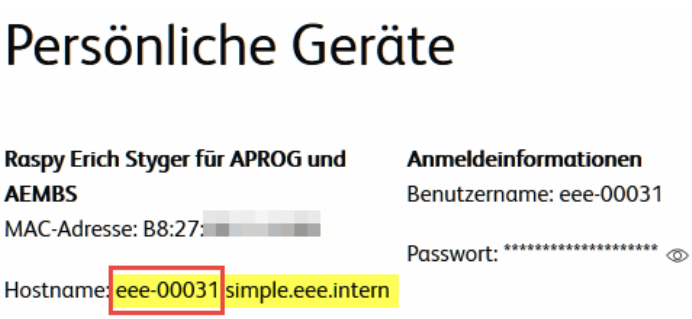


Abbildung 24: Zugeteilter Hostname

2. Ändere den Namen **ohne** simple.eee.intern in den beiden folgenden Dateien:

```
pi@raspy:$ sudo nano /etc/hostname
```

und

```
pi@raspy:$ sudo nano /etc/hosts
```

3. Anschliessend das Raspberry Pi neu starten mit

```
pi@raspy:$ sudo reboot
```

4. Nun kann man auf das Raspberry Pi mit dem Host Namen zugreifen (Abbildung 25):

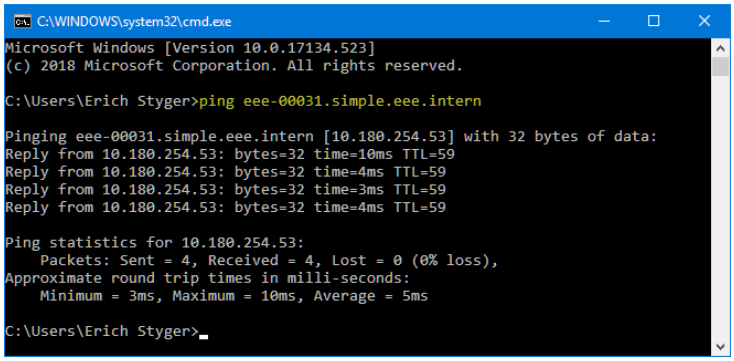


Abbildung 25: Ping mit Host Namen

5. Eventuell dauert es ein paar Minuten, bis der Host-Namen mit dem DHCP Server verfügbar ist. Bei Problemen können Sie immer die IP Adresse auf https://eeeportal.hslu.ch/ abfragen und z.B. für die SSH Verbindung verwenden.

Achtung: Unbedingt warten bis das Raspberry Pi vollständig hochgefahren ist, bevor man versucht per Hostnamen darauf zuzugreifen!!
Beachtet man dies nicht und ist das Raspberry Pi noch nicht im WLAN eingebucht, so versucht der HSLU DNS Server erst nach etwa 15 Minuten wieder das Raspberry Pi zu erreichen!

1.8 SSH Verbindung

1. Um mit SSH auf das Raspberry zugreifen zu können, müssen Sie entweder den Host-Namen z.B. `eee-12345.simple.eee.intern` oder die IP Adresse kennen. Die Namensauflösung des EEE-Netzes funktioniert wie oben erwähnt nicht immer zuverlässig. Im Fehlerfall kann direkt die IP-Adresse verwendet werden, die im EEE-Portal ersichtlich ist oder sie können die Adresse per UART Verbindung auslesen.

2. Mit 'PuTTY' (auch andere Tools sind möglich) kann nun eine SSH (Secure Socket Shell) Verbindung aufgebaut werden. In der Category das Element Session auswählen. Anschliessend den Hostnamen oder die IP Adresse mit Port *22* eintippen und eine neue Session mit *Open* (Abbildung 26) öffnen.

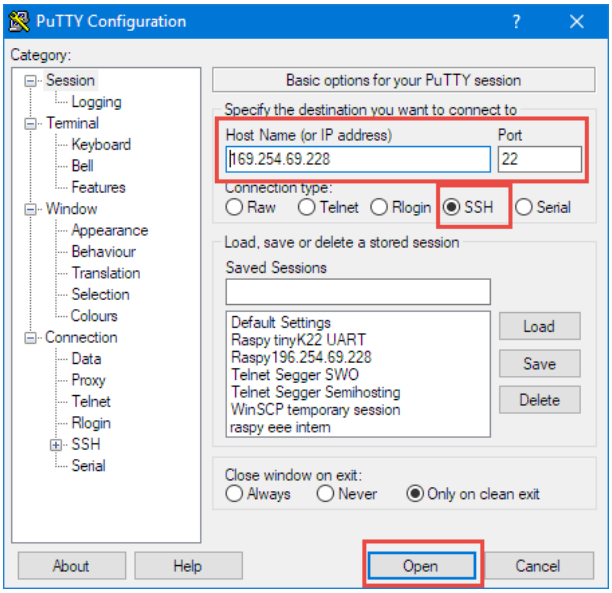


Abbildung 26: SSH Verbindung mit PuTTY

3. Alternativ kann auch die Eingabeaufforderung mit folgendem Kommando verwendet werden:

```
PC> ssh <user>@<ip>
```

4. PuTTY warnt m Ersten Verbindungsaufbau, dass der Server Host Key unbekannt sei. Diese Meldung kann ignoriert und mit Ja/Yes bestätigt werden.

1.9 Passwort

Das Passwort können Sie mit dem folgenden Befehl ändern:

```
pi@raspy:$ passwd
```

1.10 WinSCP

Ein anderes nützliches Werkzeug ist WinSCP. Damit können Sie mittels SSH auf das Dateisystem des Raspberry Pi zugreifen. (Abbildung 27).

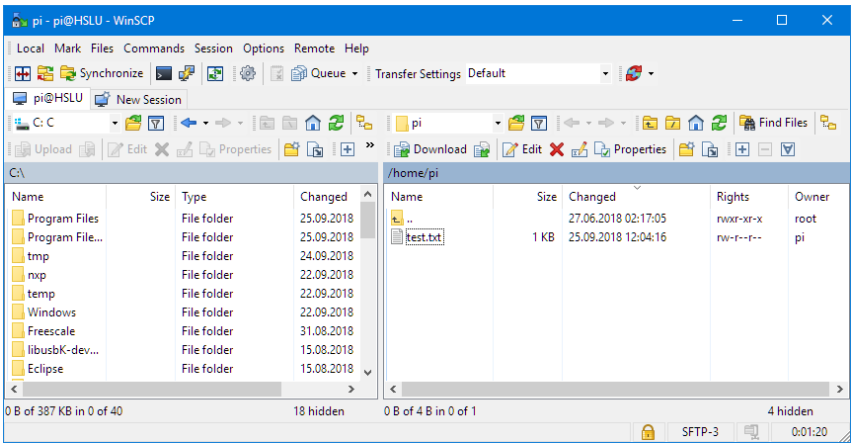


Abbildung 27: WinSCP

1.11 Updates

Da wir nun Zugriff auf das Internet haben, können wir das System mittels Updates auf den aktuellen Stand bringen. Zuerst müssen die Paketlisten aktualisiert werden:

```
pi@raspy:$ sudo apt update
```

Anschliessend kann man mit

```
pi@raspy:$ sudo apt upgrade
```

die Pakete aktualisieren. Dies kann einige Minuten dauern kann.

> **Achtung:** Starte das Update nur, wenn man auch genügend Zeit (5-30 Minuten) dazu hat. Ein laufendes Update lässt sich schlecht unterbrechen.

2 Linux Adminstration

2.1 Nützliche Linux Kommandos

Dies ist eine kleine Sammlung von hilfreichen Befehlen bezüglich dem Raspberry Pi und dem Umgang mit Linux. Probiere die Kommandos aus und brauche den `man` Befehl, um deren Bedeutung und Anwendung kennenzulernen.

Befehl	Bedeutung	Erklärung
man	manual	Hilfe zu Befehlen
pwd	print working directory	Aktuelles Verzeichnis anzeigen
cd [Pfad]	change directory	Verzeichnis wechseln, cd /home/pi, cd meinOrdner,
cd ..	Verzeichnis wechseln	Eine Hierarchie im Verzeichnis hoch
ls	list	Inhalt anzeigen von aktuellen Verzeichnis
ls -l	list long	Inhalt anzeigen mit zusätzlichen Informationen
mkdir [Name]	make directory	Ein Verzeichnis erstellen
rmdir [Name]	remove directory	Ein Verzeichnis löschen
rm [Name]	remove	Ein File löschen
rm -r	remove recursive	Rekursives Löschen
rmdir	remove directory	Löschen leeres Verzeichnis
mv [Name] [Verzeichnis]	move	Ein File verschieben
ifconfig	Interface configuration	Anzeigen der IP Adressen
sudo [Befehl]	run as super user	Als Administrator ausführen
sudo reboot		Systemneustart
uname -a	System Information drucken	Kernel Version anzeigen
touch	Zeitstempel ändern	leere Datei erstellen oder Datum aktualisieren
ping	Echo Request senden	Internetverbindung prüfen, ping www.google.com
history	vorherige Befehle auflisten	Kommando History anzeigen
!<nr>	i17	Kommando in History ausführen
!!		Letztes Kommando ausführen, oder auch !-1

Befehl	Bedeutung	Erklärung
Ctrl+C	SIGINT senden	Interrupt an eine Prozess senden um ihn zu beenden
Ctrl+Z	SIGSTOP senden	Prozess im Hintergrund weiterlaufen lassen. Mit fg und bg zwischen Vordergrund/Hintergrund umschalten.
CTRL+R	reverse-i-search	Suchen in History
clear		Konsole löschen (eigentlich Scrollen) oder CTRL+L
grep <pattern>	history | grep ls	Suchen nach Zeichen
whoami	who am I	Aktueller Benutzernamen
cp	copy	Kopieren von Dateien und Verzeichnissen
cat	concatenate	Zusammenfügen von Dateien und auf Konsole ausgeben
more	more file.txt	Seitenweise Ausgabe von Text und Dateien
less	ls -al | less	Ähnlich wie 'more', aber man kann vor- und zurückblättern
tail	Ende	Ausgabe Dateiende
which	Welches	Wo befindet sich ein Programm
kill	Signal senden	Prozess terminieren mit kill -9 <id>
ps	ps -aux	Prozesse auflisten
df	disk free	Belegung Speicherplatz
free	Freier Speicher	Belegung Memory
top	Linux Prozesse anzeigen	Taskmanager, 'q' um es zu beenden
htop	Interaktiver Prozess Viewer	'graphische' Taskliste, 'q' um es zu beenden

Für die Verwendung von `htop` ist der Artikel htop Explained Visually sehr zu empfehlen.

2.2 Backup der SD-Karte (nicht für den Zumo)

Wie auch bei anderen wichtigen Daten lohnt es sich von Zeit zu Zeit ein Backup der SD-Karte zu erstellen. Wird die SD-Karte in den Computer eingesetzt und mit dem Explorer geöffnet, sollte ein Bild gemäss Abbildung 28 erscheinen.

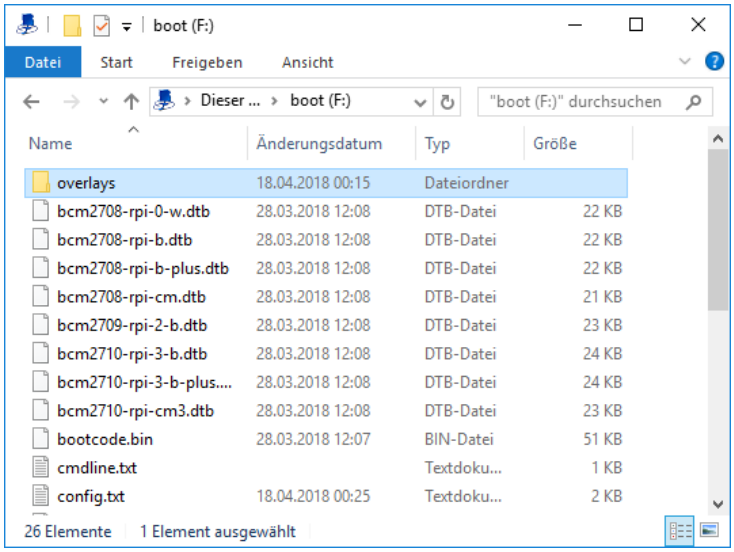


Abbildung 28: SD-Karte im Explorer

Das Angezeigte ist längst nicht der gesamte Inhalt der SD-Karte. Die meisten Dateien sind für

das Windows Betriebssystem nicht lesbar. Der Inhalt kann somit nicht mit `Ctrl`+`C` und dann `Ctrl`+`V` kopiert werden. Zum Beschreiben und Lesen des kompletten Inhaltes kann Beispielsweise die Software Win32 Disk Imager verwendet werden.

1. Den Datenträger mit der SD-Karte auswählen.
2. Den Dateinamen für das Backup mit der Endung `.img` angeben.

   > **Hinweis:** Gib eine Datei auf dem Host/Laptop an, in die das Backup gespeichert werden soll.

3. Auf *Lesen* klicken.

Der komplette Inhalt der SD-Karte wird nun an den Zielort kopiert.

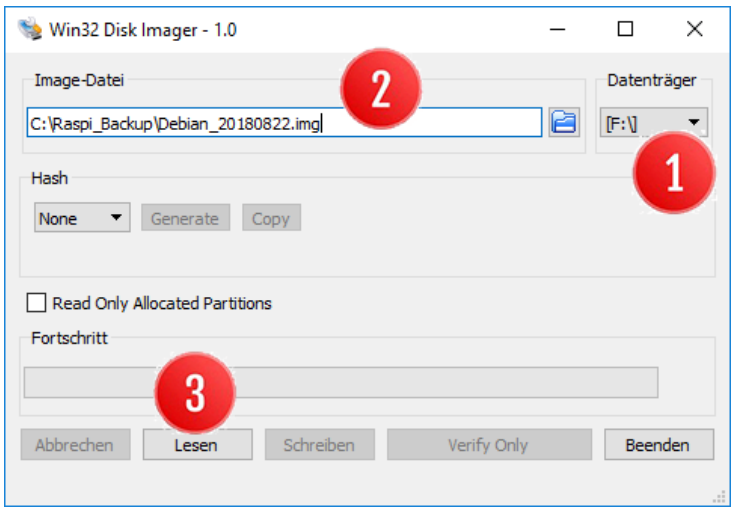


Abbildung 29: Inhalt speichern

Schritte zum Wiederherstellen des Backups auf die SD Karte:

1. Die SD-Karte auswählen.
2. Die gewünschte `.img` Datei wählen. Dazu kann der blaue Button gewählt werden.
3. Bei einem gültigen File ist der Button *Schreiben* nicht mehr ausgegraut und kann gewählt werden.

Ein Image wird nun auf die SD-Karte geschrieben.

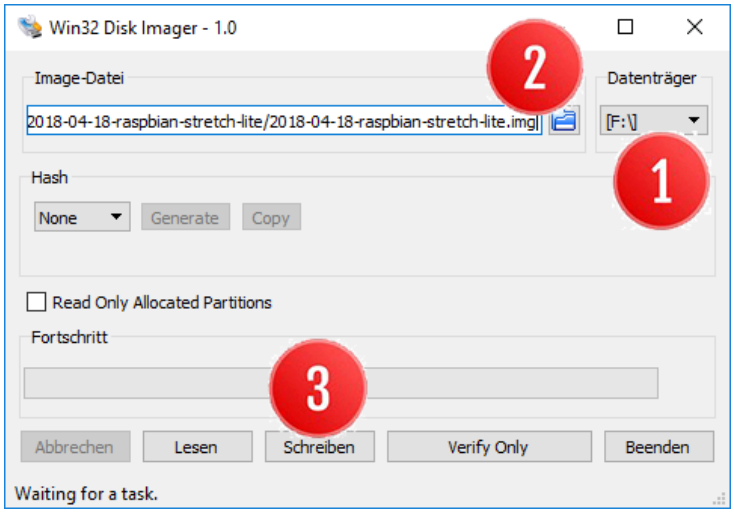


Abbildung 30: Inhalt speichern

2.3 History löschen

Alle eingegebenen Befehle werden in einer History der Shell gespeichert. Gibt man aus Versehen ein Passwort statt eines Befehls in der Shell ein, so kann auch unser Passwort in der History gespeichert werden. Sie befindet sich im `.bash_history` innerhalb des Benutzerverzeichnisses. Der Inhalt kann man mit

```
pi@raspy:$ more ~/.bash_history
```

anschauen. Entweder editiert man die Datei und entfernt die geheimen Informationen oder mit

```
pi@raspy:$ rm ~/.bash_history
```

kann man die History komplett löschen, gefolgt von

```
pi@raspy:$ touch ~/.bash_history
```

das eine neue leere Datei erstellt.
Zusätzlich sollte man die History löschen mit

```
pi@raspy:$ history -c
```

gefolgt vom Abmelden des Benutzers:

```
pi@raspy:$ logout
```

3 SSH Verbindung ohne Passworteingabe

In diesem Teil erstellen wir die benötigten Konfigurationen, um eine sichere SSH Verbindung zwischen Host PC und der Linux Maschine herzustellen, ohne immer ein Passwort eingeben zu müssen.

3.1 Schlüssel auf Linux Seite generieren

Zuerst konfigurieren wir die Linux-Seite. Hierzu müssen wir als normaler Benutzer auf dem Raspberry eingeloggt sein.
Die Einstellungen werden im Benutzerverzeichnis in der versteckten `.ssh` Verzeichnis gespeichert. Falls dieses Verzeichnis noch nicht existiert, erstellen Sie es im zuoberst im Benutzerverzeichnis:

```
pi@raspy:$ mkdir ~/.ssh
```

Aus Sicherheitsgründen wollen wir nur uns selber Zugriff auf dieses Verzeichnis geben:

```
pi@raspy:$ chmod 700 ~/.ssh
```

Es ist wichtig, dass nur der Benutzer die Dateien in diesem Verzeichnis lesen und ändern kann. Ansonsten könnte jemand anders seinen public Key hier eintragen und sich dann in unser Konto einloggen.
Als nächstes erstellen wir eine leere Datei für die genehmigten Schlüssel, falls diese noch nicht existiert:

```
pi@raspy:$ touch ~/.ssh/authorized_keys
```

Auch hier sollten wir die Berechtigungen einschränken, damit niemand sonst die Datei lesen kann:

```
pi@raspy:$ chmod 600 ~/.ssh/authorized_keys
```

Als nächstes generieren wir die Schlüssel. Hierzu wechseln wir in das .ssh Verzeichnis unseres Benutzers:

```
pi@raspy:$ cd ~/.ssh
```

Dort erstellen wir einen RSA Schlüssel mit 4096 Bits:

```
pi@raspy:$ ssh-keygen -m PEM -b 4096 -t rsa -f piKey
```

> **Hinweis:** Auf einem Raspberry Pi 4 muss zwingend die Option **-m PEM** im 'ssh-keygen **-m PEM** -b 4096 -t rsa -f piKey' verwendet werden, ansonsten funktioniert der automatische Upload vom Visual Studio mit dem 'SecureUpload.exe' nicht.

Die Generierung des Schlüssels dauert eine Weile, weil der Generator zufällig Informationen auswertet und für den Schlüssel verwendet.

```
Generating public/private rsa key pair.
```

Den Schlüssel könnte man mit einem Passwort sichern, was wir aber nicht machen, weil wir sonst immer wieder das Passwort eingeben müssen. Deshalb beantworten wir die folgenden Fragen mit der `ENTER` Taste:

```
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

Die Schlüssel werden in zwei Dateien angelegt, deren Namen wir angegeben haben:

```
Your identification has been saved in piKey.
Your public key has been saved in piKey.pub.
```

Zum Schluss der Hash und wird ein 'randomart' Bild erstellt:

```
The key fingerprint is:
SHA256:ru7KpCbYE6U2f6ElWs56NMmKV7yrkoCRGUwHtN0U3iY pi@raspy
The key's randomart image is:
+---[RSA 4096]----+
|++.. o.          |
|..+ + .          |
| = . E o         |
|+   . o          |
|.. = .  S        |
|o = X o.         |
|o= %.* ..        |
|* B+B ..         |
| =+=+*+          |
+----[SHA256]-----+
```

Dieses Bild ist für Menschen einfacher zu vergleichen als der komplizierte SH256 Hash Schlüssel.
`ssh-keygen` hat für uns zwei Dateien erstellt:

1. **piKey**: den geheimen Private Key
2. **piKey.pub**: den öffentlichen Public Key

Den Public Key können wir nun in die Datei `authorized_keys` innerhalb des `.ssh` Verzeichnisses im 'home' des Benutzers (pi) schreiben:

```
pi@raspy:$ cat piKey.pub >> authorized_keys
```

Falls die `authorized_keys` noch nicht besteht, muss man anstatt `>>` (für Anhängen) die Umleitung mittels `>` (für Erstellen) verwenden.

3.2 Windows Host Konfiguration

Die nächsten Schritte werden auf dem Windows Rechner ausgeführt. Hierzu benutzen wir eine Kommandozeilen-Konsole (Abbildung 31), z.B. mit `WINDOWS-TASTE` + `R`, und dann `cmd` eintippen.

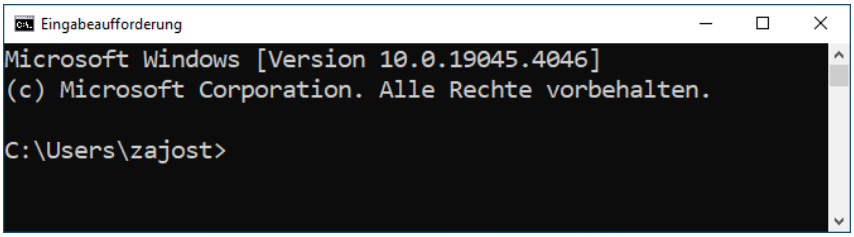


Abbildung 31: Windows 10 User Prompt

In unserem Benutzerverzeichnis (normalerweise "C:\Users\<user>") erstellen wir analog ein Verzeichnis mit Namen `.ssh`, falls dieses noch nicht existieren sollte:

```
PC> mkdir "%USERPROFILE%\.ssh"
```

In dieses Verzeichnis kopieren wir nun mit `scp` (Secure Copy) unseren Private Key vom Raspberry Pi, wobei natürlich anstatt 'hostname' der aktuelle Hostnamen verwenden wird (Achtung: zwischen ':' und '.ssh' ist kein Leerzeichen.):

```
PC> scp pi@hostname:.ssh/piKey "%USERPROFILE%\.ssh"
```

Möglicherweise wird eine Warnung angezeigt, das die Authentizität der anderen Maschine nicht überprüft werden kann, was möglicherweise ein 'man in the middle' sein könnte. Mit 'yes' bestätigen wir, dass wir der Verbindung vertrauen und werden nach dem Verbindungspasswort gefragt.
Nach dem Kopieren des Private Key brauchen wir den nicht mehr auf dem Raspberry Pi, somit kann er gelöscht werden.

> **Info:** Unter Umständen funktioniert `scp` nicht zuverlässig. Alternativ können Sie das Programm WinSCP verwenden, das wir früher kennengelernt haben. Mit `CTRL`-`ALT`-`H` können versteckte Dateien angezeigt werden.

Als nächstes wollen wir die SSH Verbindung vereinfachen, indem wir eine SSH Konfigurationsdatei verwenden. Im `.ssh` Verzeichnis auf dem Host erstellen Sie eine Datei (z.B. mit Notepad) namens `config`, mit folgendem Inhalt:

> **Hinweis:** Ersetzen Sie die Stellen mit <...> mit den Angaben ihres Rechners. Ungültige Eintrage müssen mit # auskommentiert werden, ansonsten können Fehler auftreten.

```
# Raspberry Pi an der HSLU
Host pi-hslu
  hostName eee-<??>.simple.eee.intern
  User pi
  IdentityFile "C:\Users\<UserName>\.ssh\piKey"
```

> **Hinweis:** Sie müssen dabei den Benutzernamen (<UserName>) auf dem Host PC und IP oder Hostnamen für Ihr Raspberry Pi verwenden.

3.3 Testen der Verbindung

Nun können Sie die Verbindung zum Raspberry testen mit

```
PC> ssh pi-hslu
```

Dabei sollten Sie nicht nach einem Passwort gefragt werden, da Sie ja den Private Key auf dem Host haben.
Sollten Sie eine Meldung wie untenstehend bekommen, dann können Sie einfach die angegebenen Zeilen in der Datei `known_hosts` im .ssh Verzeichnis löschen:

```
Warning: the ECDSA host key for 'raspy' differs from the key for
the IP address '192.168.0.118'
Offending key for IP in C:\Users\<User>/.ssh/known_hosts:9
Matching host key in C:\Users\<User>/.ssh/known_hosts:15
```

3.4 SSH ohne Passwort mit PuTTY

Man kann auch das PuTTY so konfigurieren, dass man nicht immer nach dem Benutzernamen und Passwort gefragt wird. Allerdings muss man für PuTTY den Schlüssel erst in das PuTTY eigene Format (`.ppk`) konvertieren. Dazu verwenden wir das Programm `puttygen`, das sich im gleichen Verzeichnis wie das Programm PutTTY befindet.
Hierzu startet man den KeyGen des PuTTY und importiert den Private Key (Abbildung 32).

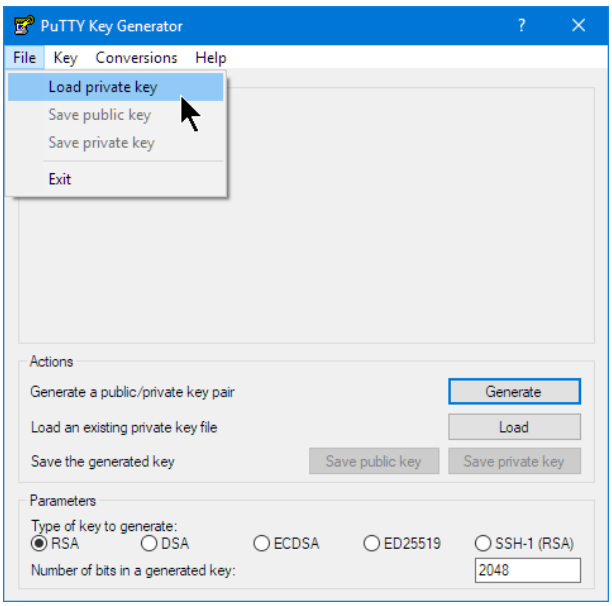


Abbildung 32: PuTTY Keygen

Lade den *Private* Key (z.B. den `piKey`) (Abbildung 33).

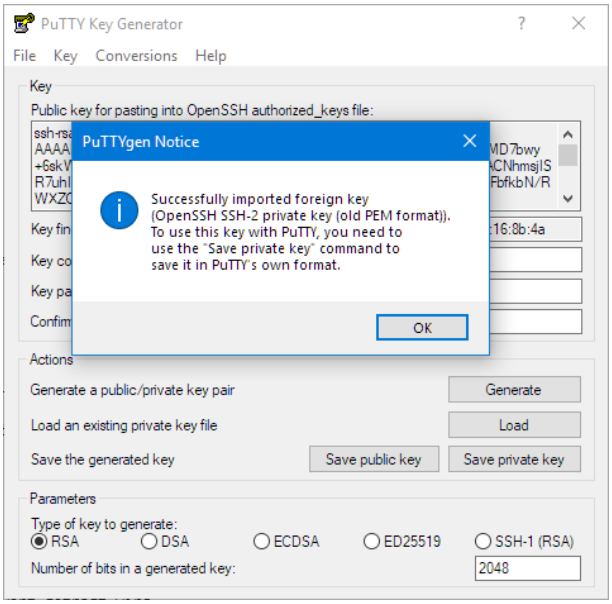


Abbildung 33: PuTTY Keygen

Dann den Schlüssel abspeichern (Abbildung 34).

Achtung: Es muss unbedingt die Dateierweiterung `.ppk` verwendet werden, damit nicht ein existierender Schlüssel überschrieben wird!

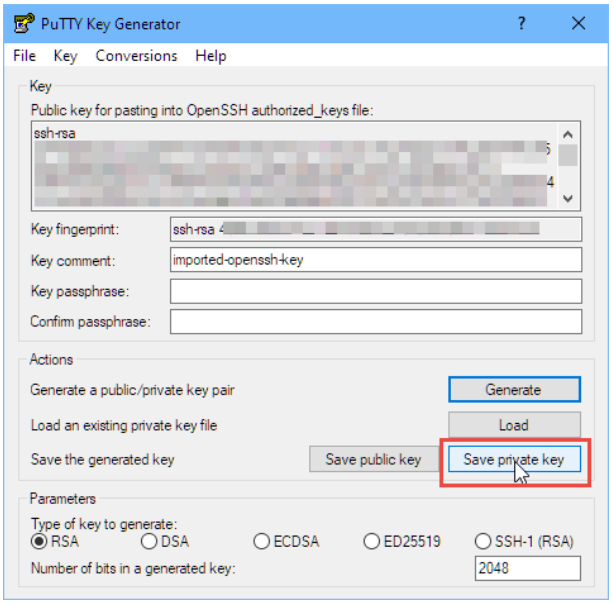


Abbildung 34: PuTTY Keygen

Nun trägt man unter Connection den Benutzernamen für das automatische Login ein (Abbildung 35).

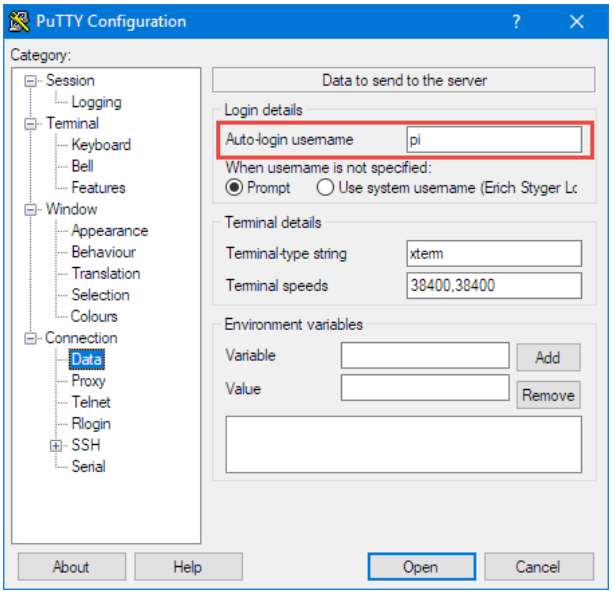


Abbildung 35: PuTTY Connection Auto Username

Bei 'Auth' zeige auf die die erstellte `.ppk` Datei (Abbildung 36).

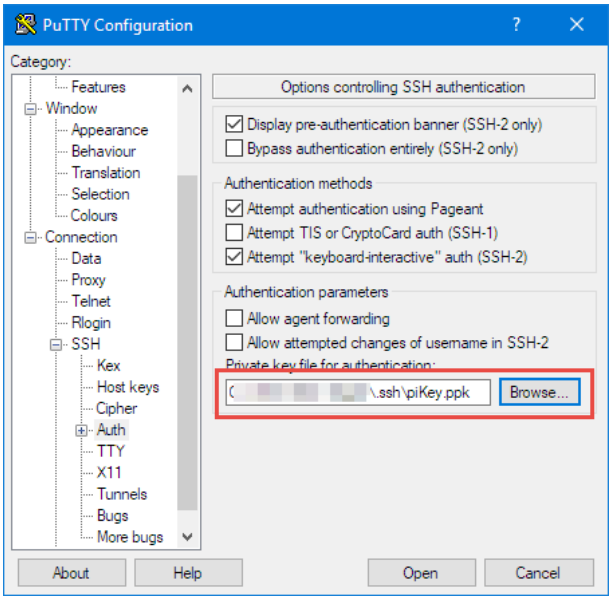


Abbildung 36: PuTTY Keygen

Nach dem Speichern der Einstellung kann man sich mit PuTTY ohne Benutzername und Passwort einloggen.

4 .NET Cross Development Installation auf Linux

Bis jetzt haben wir nur auf dem Host PC entwickelt. Nun möchten wir .NET benutzen, um die Anwendungen auf dem PC und auch auf dem Linux System laufen zu lassen. Wenn man auf dem PC für eine andere Zielplattform entwickelt, nennt man dies auch *Cross-Platform Development*. Im Folgenden werden wir die nötige Laufzeitumgebung auf dem Raspberry Pi einrichten. Im zweiten Schritt benutzen wir eine SSH Verbindung, um vom Windows PC die Anwendung auf das Linux Target zu übertragen.

4.1 Installation von .NET auf Linux

Für die Installation müssen nachfolgende Befehle auf dem Raspberry Pi ausgeführt werden:

> **Hinweis:** Es kann sein, dass beim ersten Ausführen von .NET Anwendungen die Runtime noch zusätzliche Dateien vom Internet lädt. Es sollte daher auch nach der Installation eine funktionierende Internetverbindung vorhanden sein.

1. Mit dem Package Manager *apt* ein Update der Pakete machen. Damit wird die Liste der Softwarepakete und deren Version geholt.

```
pi@raspy:$ sudo apt update
```

2. Mit dem Package Manager *apt* anschliessend ein Upgrade der Pakete machen. Dies lädt und installiert die neuen Pakete.

```
pi@raspy:$ sudo apt upgrade
```

3. Mit dem Package Manager *apt* die notwendigen Pakete installieren: `curl`, `gettext` und `libunwind8`:

```
pi@raspy:$ sudo apt install curl libunwind8 gettext
```

4. Als nächstes müssen wir die Datei für das .NET SDK und die Runtime finden. Die Releases sind auf GitHub: release-notes. Die Release Notes beinhalten .md Dateien mit den Links zu den Downloads.

5. Wir verwenden die Version 10.0 welche wir auf GitHub .NET 10.0 herunterladen können. Damit wir ein Programm auf dem Linux laufen lassen können, muss die gleiche Version installiert sein, sonst bekommt man eine Fehlermeldung. Wir benötigen das ARM64 SDK[1] Binaries (Abbildung 37). Direkter Download Link

OS	Installers	Binaries
Linux	Package manager instructions	Arm32 | Arm32 Alpine | Arm64 | Arm64 Alpine | x64 | x64 Alpine
macOS	Arm64 | x64	Arm64 | x64
Windows	Arm64 | x64 | x86 | winget instructions	Arm64 | x64 | x86
All	dotnet-install scripts	

Abbildung 37: .NET ARM SDK Binaries

[1] Da es sich beim Raspberry OS Image um ein 64bit Linux handelt, können wir die 64bit Binaries verwenden.

6. Laden Sie das SDK nicht lokal auf Ihren PC herunter, sondern kopieren Sie nur den Link in die Zwischenablage. Loggen Sie sich beim Raspberry Pi ein und laden Sie das SDK mittels `wget` herunter. Am besten ins eigene Home-Verzeichnis, da man hier Schreibzugriff hat. Der Dateiname lautet `dotnet-sdk-X.X.XXX-linux-arm.tar.gz`

   ```
   pi@raspy:$ wget <Download URL von der Webseite>
   ```

   Falls Sie die Datei unter Windows heruntergeladen haben, können Sie sie beispielsweise mit WinSCP auf das Raspberry Pi umkopieren.

7. Erstellen Sie ein Verzeichnis für die Installation von .NET:

   ```
   pi@raspy:$ sudo mkdir /opt/dotnet
   ```

8. Extrahieren Sie alle Dateien in das soeben erstellte Verzeichnis:

   ```
   pi@raspy:$ sudo tar -xvf dotnet-sdk-10.0.103-linux-arm64.tar.gz -C
   /opt/dotnet/
   ```

9. Anschliessend muss ein symbolischer Link erstellt werden, damit die ausführbare Datei 'dotnet' gefunden werden kann:

   ```
   pi@raspy:$ sudo ln -s /opt/dotnet/dotnet /usr/local/bin
   ```

   Wenn Sie einen Befehl eintippen, wird automatisch auch im Verzeichnis '/usr/local/bin' nach Binaries gesucht wird:
   `ls -al /usr/local/bin`
   ergibt
   `lrwxrwxrwx 1 root staff 18 Oct 8 11:28 dotnet -> /opt/dotnet/dotnet`

10. Für das .NET SDK muss zusätzlich noch eine Umgebungsvariable `DOTNET_CORE` gesetzt werden, welche auf die Laufzeitumgebung (Runtime) zeigt. Ansonsten kann .NET unter Umständen die benötigten Bibliotheken nicht finden. Mit `export` wird die Variable gesetzt:

    ```
    pi@raspy:$ export DOTNET_ROOT=/opt/dotnet
    ```

    Mit `env` können Variablen abgefragt werden:

    ```
    pi@raspy:$ env | grep DOT
    ```

    ```
    export DOTNET_ROOT=/opt/dotnet
    ```

11. Da das `export` Kommando die Variable nur für die aktuelle Session setzt, müsste man dies bei jedem Login neu machen, was umständlich wäre. Da beim Login das Skript `.profile` ausgeführt wird, können wird dies dort eintragen:

    ```
    pi@raspy:$ nano .profile
    ```

    ```
    # Set .NET Core environment variable
    export DOTNET_ROOT=/opt/dotnet
    export DOTNET_CLI_TELEMETRY_OPTOUT=true
    ```

    Mit DOTNET_CLI_TELEMETRY_OPTOUT' teilen Sie der Laufzeitumgebung mit, dass keine anonymen Statistikdaten erfasst und ins Internet übertragen werden sollen.

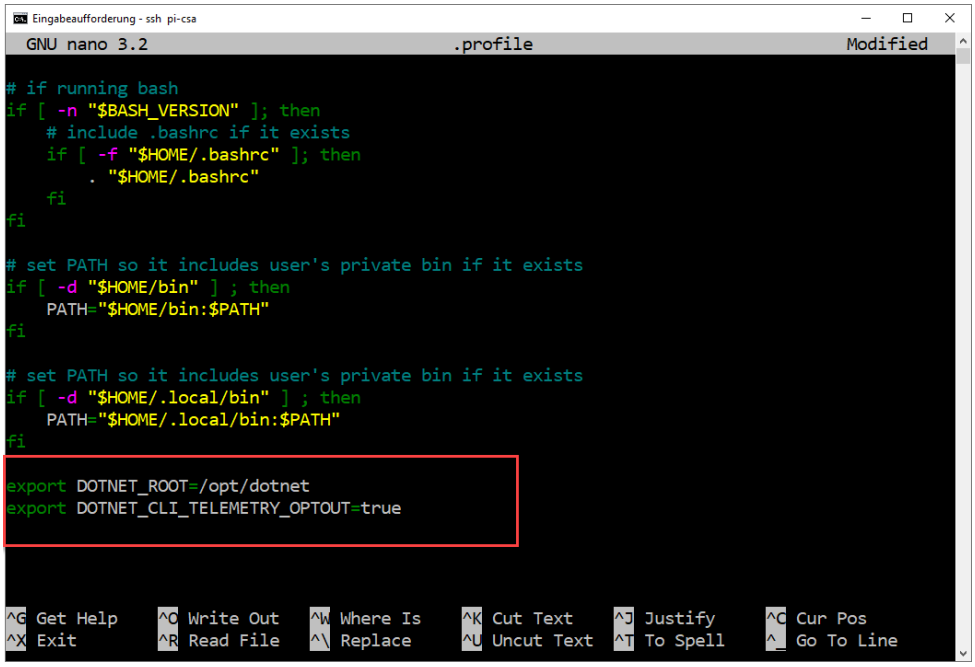


Abbildung 38: Setzen von DOTNET_ROOT in .profile

Damit wird diese Variable bei jedem neuen Login gesetzt.

12. Überprüfe, ob die Installation erfolgreich war:

```
pi@raspy:$ dotnet --help
```

Die Ausgabe sollte ähnlich der Abbildung 39 sein.

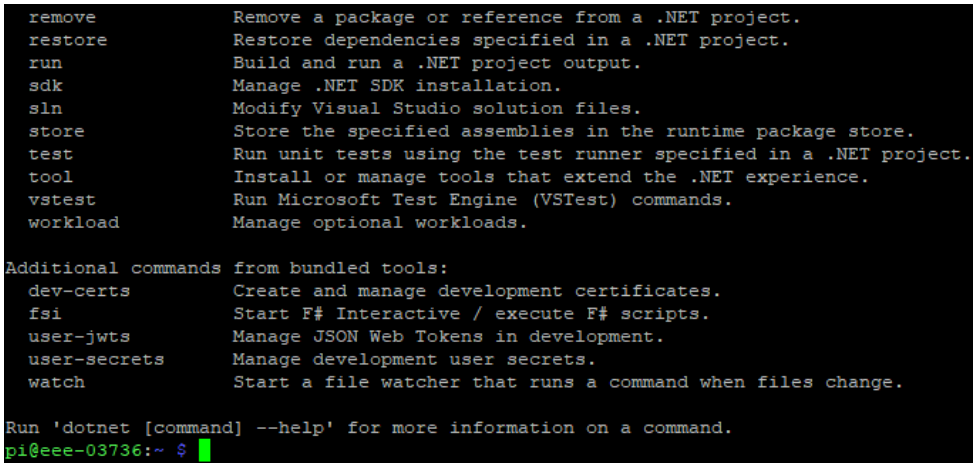


Abbildung 39: Befehl dotnet –help

13. Optional kann das heruntergeladene File entfernt werden:

```
pi@raspy:$ rm dotnet-sdk-X.X.XXX-linux-arm.tar.gz
```

14. Nun können wir ausprobieren, ob wir auf dem Raspberry Pi eine .NET Anwendung erstellen können:

```
pi@raspy:$ dotnet new console -o myapp
```

15. Wechseln in das erstellte Verzeichnis für die Anwendung:

```
pi@raspy:$ cd myapp
```

16. Und starten Sie die Anwendung:

```
pi@raspy:$ dotnet run
```

17. Nach einer Weile erfolgt dann die Ausgabe des Programmes (Abbildung 40).

```
pi@eee-00031:~/myapp $ cat Program.cs
 using System;

namespace myapp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
pi@eee-00031:~/myapp $
```

Abbildung 40: myapp Anwendung auf Raspy

Es dauert eine Weile (bis zu 1 Minute), bis die .NET Runtime geladen und die Anwendung kompiliert ist.

> **Info:** Der Befehl `dotnet run` wird benötigt, um die Anwendung zu kompilieren und die `.dll` zu erstellen.

18. Wesentlich schneller geht es, wenn man das kompilierte Binary im `bin` Verzeichnis direkt ausführt:

```
pi@raspy:$ dotnet bin/Debug/net8.0/myapp.dll
```

Hiermit ist das .NET auf dem Linux System installiert und bereit.